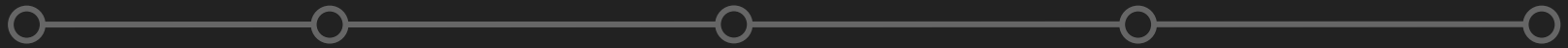
Intro to Clustering with K-Means

Unsupervised Learning



GENERAL ASSEMBLY

Lesson roadmap



Welcome + warm-up

Unsupervised Learning & Clustering Use Cases

How K-means works

Hands-on K-means & Evaluation

Bring It Home

Learning objectives



1. Know the difference between supervised and unsupervised learning.
2. Understand when to use clustering.
3. Perform a K-Means clustering analysis.
4. Evaluate clusters for fit with Silhouette Coefficient and Inertia.



Warm Up

What makes things belong together?

© 2021 General Assembly

Grouping items

- How would you group these objects?
- What features would you use to decide which group they belong in?













 © 2024 General Assembly

Unsupervised Learning

Supervised vs. Unsupervised Learning

Most machine learning models fall into one of two categories:

- **Supervised learning** - Given X, can we predict Y?
- **Unsupervised learning** - What does X look like, *really?* There is no Y.

 © 2025 General Assembly

Unsupervised Machine Learning

This is used for finding **clusters**, **groups** and **patterns** in datasets aka **feature discovery**.

It's also used for **dimensionality reduction**, or reducing the number of variables in a dataset without losing useful information.

We're **not** explicitly predicting something.

The results of unsupervised learning often involve a lot of **domain expertise** and **human interpretation**.

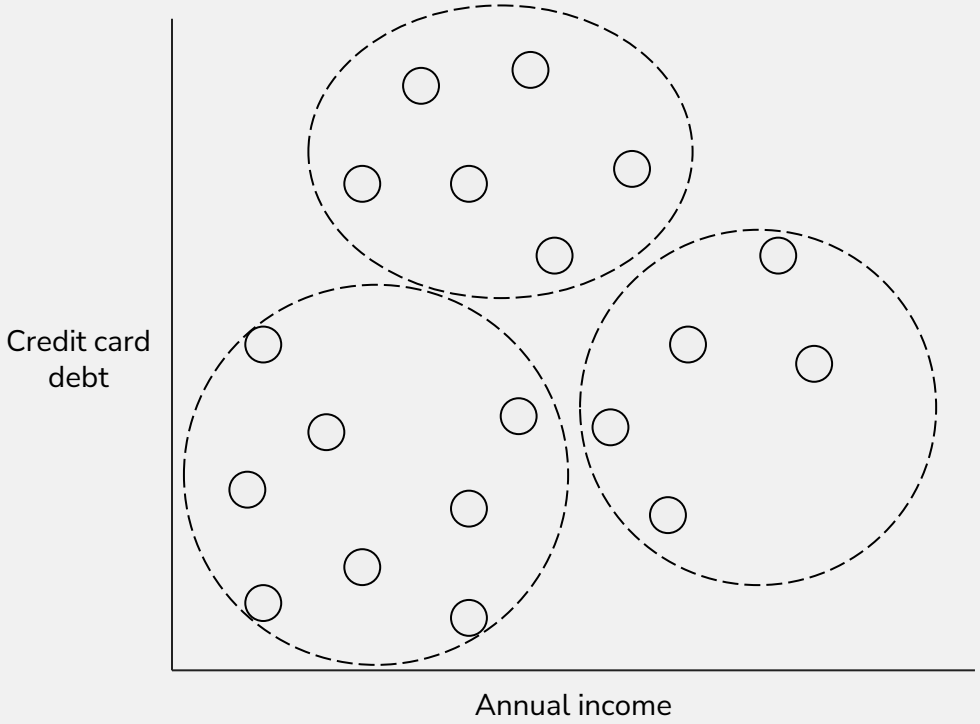
 © 2025 General Assembly

Unsupervised Machine Learning

- There is **no clear objective**
- There is **no "right answer"** (which means it's hard to tell how well you're doing)
- There's **no response variable**, only observations with features
- **Labeled data is not required**

 © 2025 General Assembly

Unsupervised Learning



Problem: Find meaningful clusters in customer data.

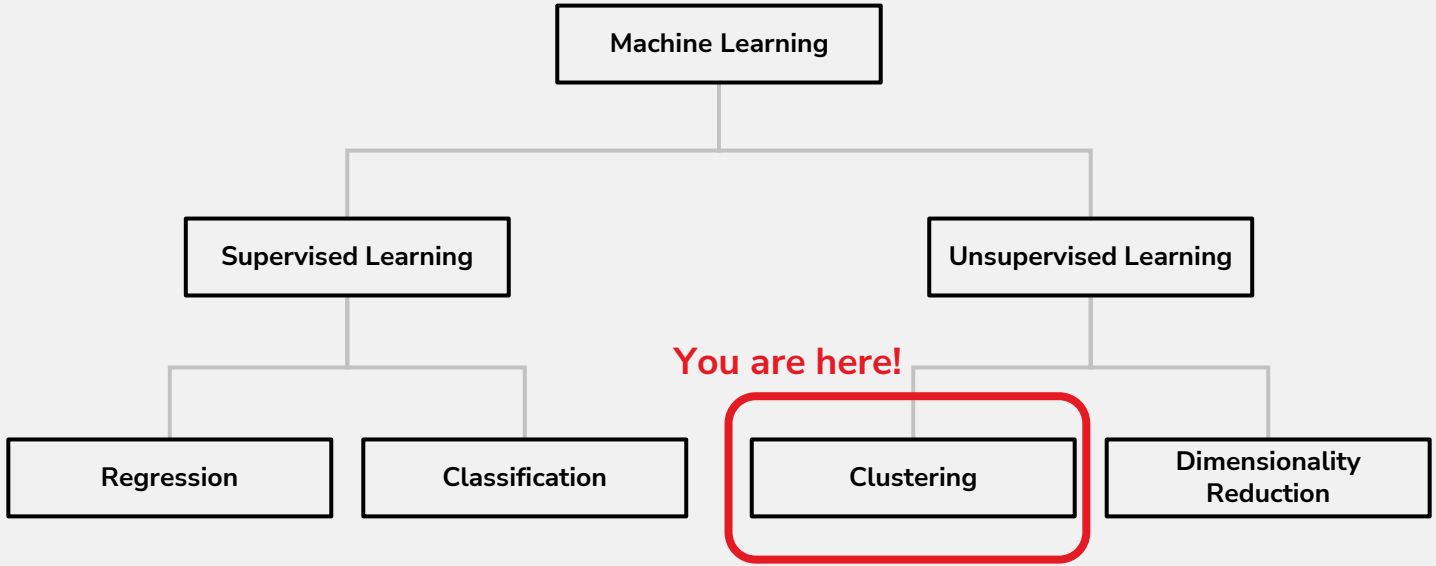
Use an **unsupervised** clustering method to find groupings in the data.

In this case, the algorithm has found three **clusters**.

Human interpretation is needed to make sense of what these clusters mean.

 © 2025 General Assembly

Roadmap of ML



 © 2025 General Assembly

Introduction to Clustering

What is Clustering?

In unsupervised learning, one strategy is to **cluster** observations into **homogeneous groups**.

Observations of the same group are more similar than observations in different groups.

Remember since this is unsupervised, we don't decide in advance what these clusters will look like. It's all up to the algorithm!

 © 2025 General Assembly

DISCUSSIONS: What is clustering used for?

What is clustering used for?

Can you think of examples when clustering data might be useful?

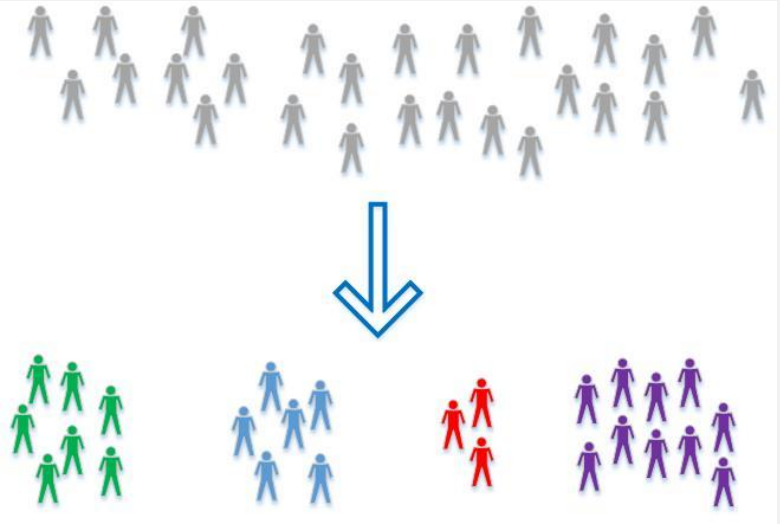
 © 2024 General Assembly

Market Segmentation

Clustering your user base is a popular use case for clustering.

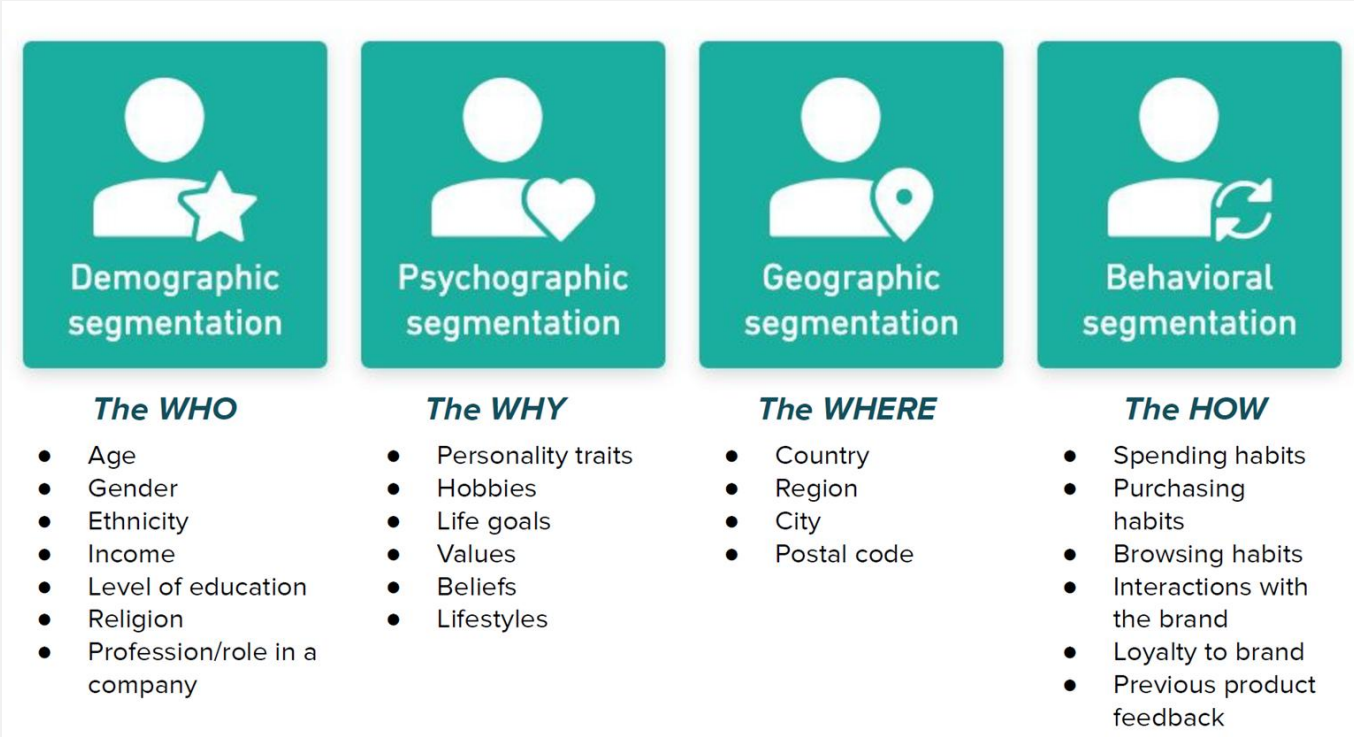
Suppose after clustering, you realize people in one cluster aren't buying your product as often as other clusters.

Why? What does this cluster look like?



 © 2025 General Assembly

Market Segmentation

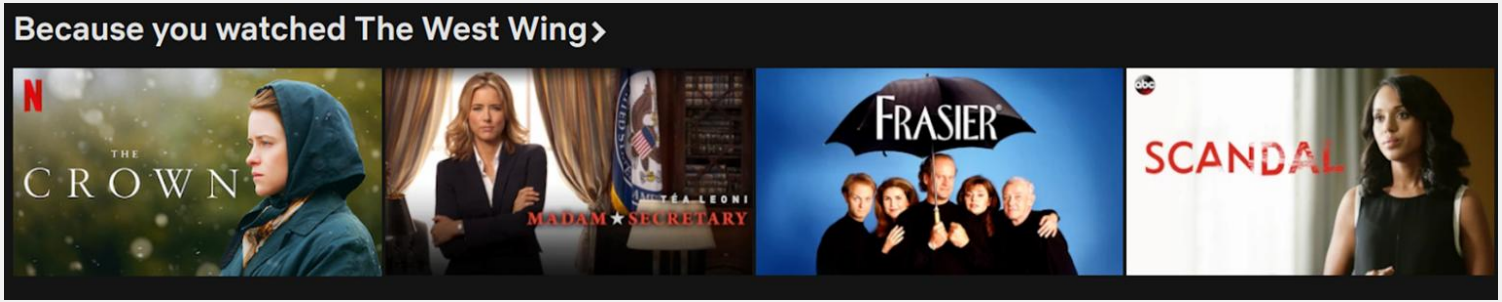


Source

 © 2025 General Assembly

Recommender Systems

Many recommender systems use clustering to filter their user base (you) into clusters. What other TV shows are other people in your cluster watching?



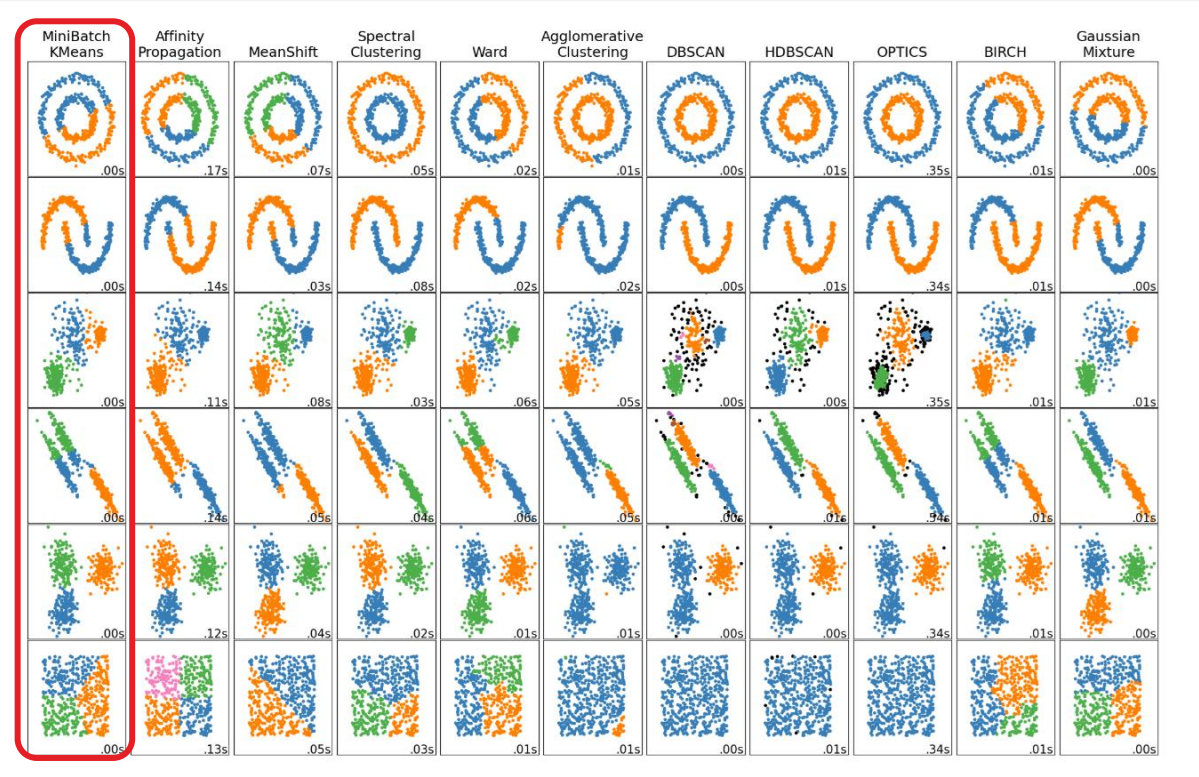
 © 2025 General Assembly

And that's not all... Additional Examples of Clustering

- Discovering meaningful categories for your data
- Reducing the number of classes by grouping (e.g. apples, pears -> fruit)
- Reducing the dimensions of a problem
- Pre-processing! Creating features or labels for supervised learning (e.g. adding the cluster as a new feature, or **pseudo-labelling** unlabelled data)
- Understanding complex systems

 © 2025 General Assembly

There are many clustering approaches, even in `scikit-learn`



 © 2025 General Assembly

Source

K-Means Clustering

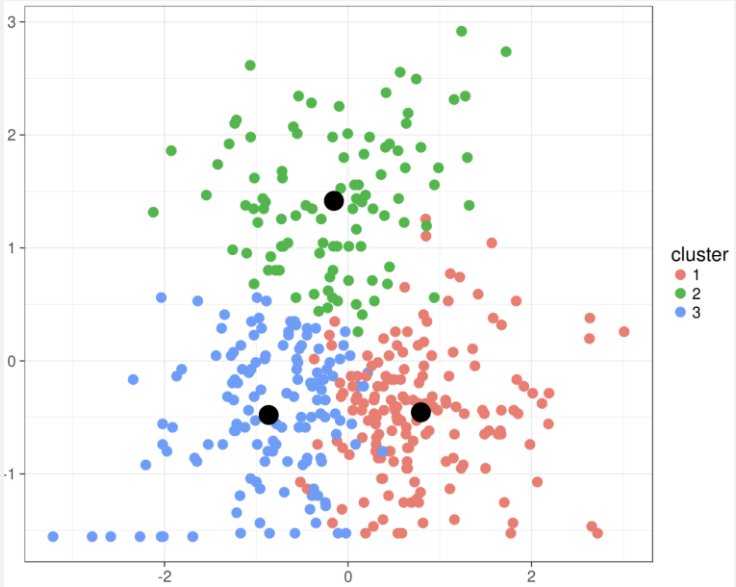
What is K-Means?

An unsupervised learning method for finding structure and groups in datasets.

Cluster data into '**k**' number of groups. This is the only thing we need to decide in advance.

Each cluster has a cluster centroid (or 'mean').

Each point in the dataset belongs to the cluster whose centroid it is closest to.



 © 2025 General Assembly

How does K-Means work?

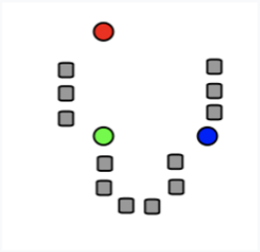
1. Pick a value for **k** (the number of clusters to create)
2. Initialize **k** centroids (starting points) in your data
3. Create your clusters by assigning each point to the nearest centroid
4. Improve your clusters: move each centroid to the center of its (new) cluster
5. Repeat Steps 3 and 4 until your centroids converge

K-Means seeks to minimize the sum of squares of each point about its cluster centroid.

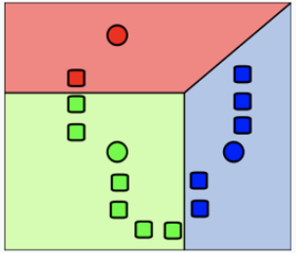
Note: Unfortunately, there's no single way to determine the absolute best number of clusters. Unsupervised learning is inherently subjective! We can, however, choose the "best" based on predetermined criteria.

 © 2025 General Assembly

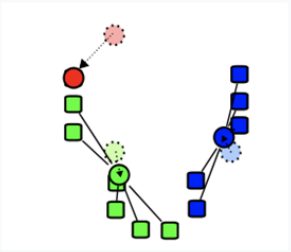
How does K-Means work?



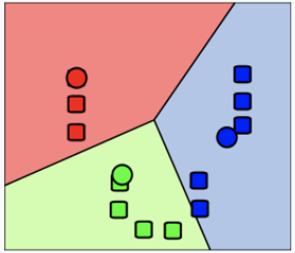
1. *k* initial "means" (in this case *k*=3) are randomly generated within the data domain (shown in color).



2. *k* clusters are created by associating every observation with the nearest mean. The partitions here represent the Voronoi diagram generated by the means.



3. The centroid of each of the *k* clusters becomes the new mean.



4. Steps 2 and 3 are repeated until convergence has been reached.

Source

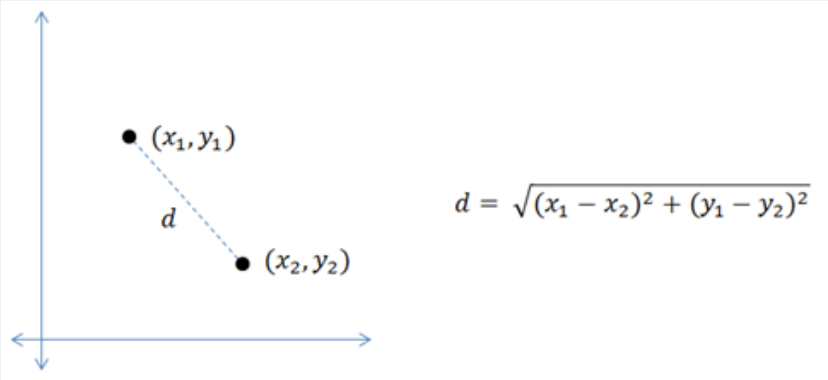
 © 2025 General Assembly

Refresher: How do we define ‘nearest’?

We measure the distance between data points using the **Euclidean** or **straight line** distance.

You might remember calculating the Euclidean distance as part of Pythagoras’ theorem at school. This is a **2D** example.

We can also calculate the Euclidean distance between points with more than two dimensions.



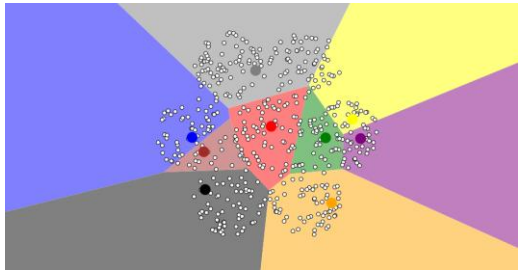
 © 2025 General Assembly

K-means Applet

Naftali Harris, CEO of SentiLink, wrote this incredible applet that illustrates how the *k*-means algorithm works.

Let's spend some time exploring it to build your intuition of *k*-means.

Click here!





SentiLink

 © 2025 General Assembly

3 questions arise from what we have looked at so far

1. How are centroids initialized?
2. How do we choose the number of clusters (k)?
3. How do we know how good our clusters are?

 © 2025 General Assembly

So... how are centroids initialized?

Three options (you can determine this using the '`init`' parameter in your model):

1. Default (`kmeans++`). This initializes the centroids to be generally distant from each other.
2. Randomly (`random`)
3. Manually (`ndarray`). You can pass this argument and provide initial centers.

Note: Manual is recommended if you know your data well enough to see the clusters without much help, but it is rarely used in practice.

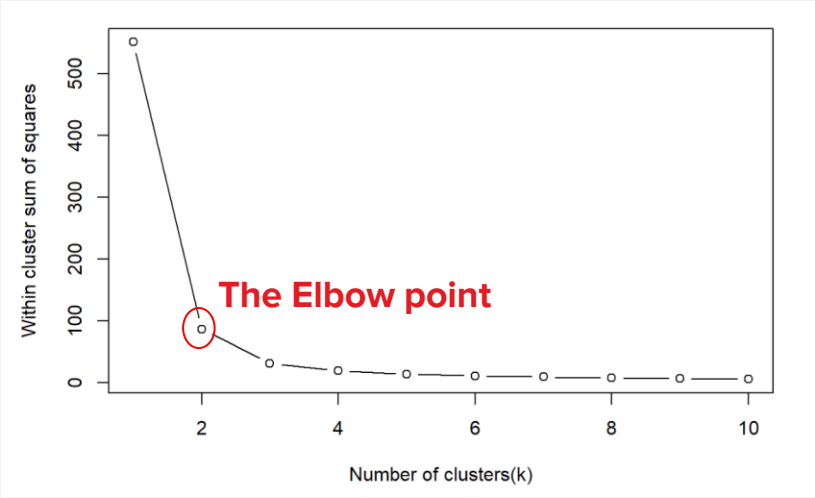
 © 2025 General Assembly

Choosing '*k*'

We may know how many clusters we have, for example customer segments.

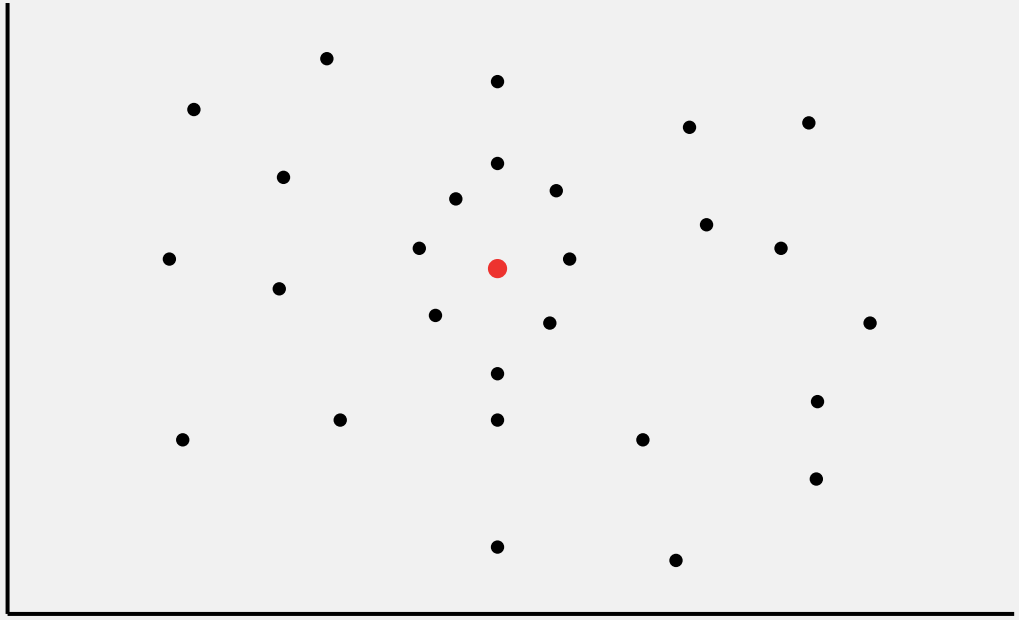
We can also select the number of clusters by changing the value of **k**, and looking at the effect this has on **how tightly packed each cluster is**.

We do this by measuring the **within-cluster sum of squares**, also known as **inertia**.



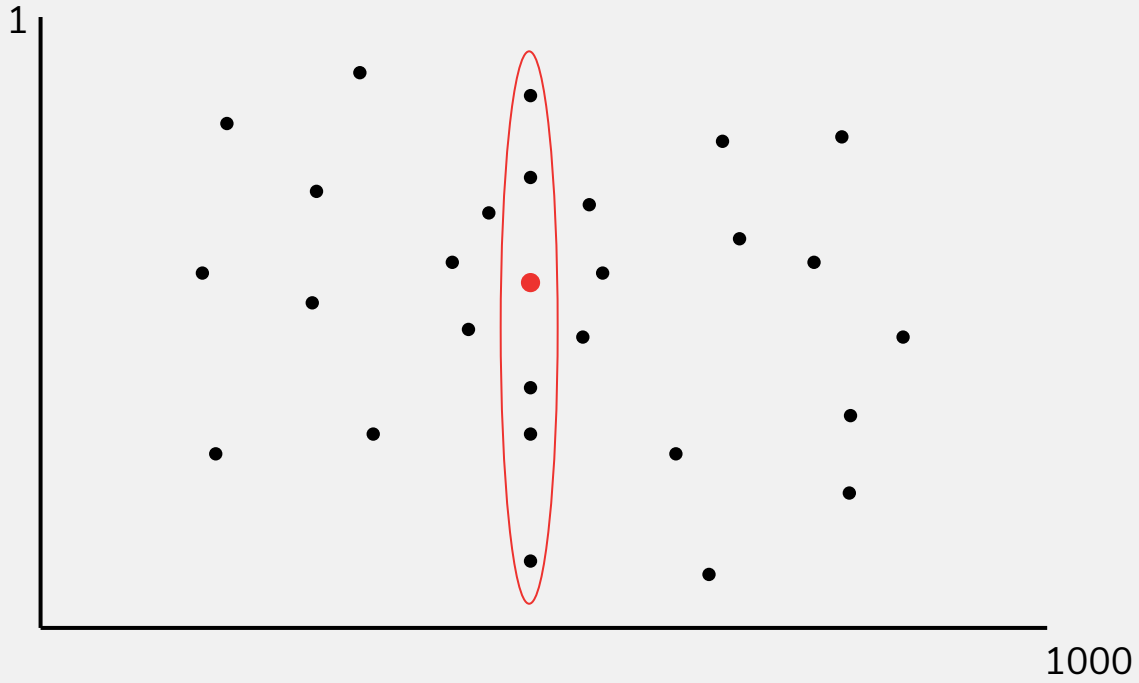
 © 2025 General Assembly

One more thing...What are the red point's 5 nearest neighbors?



 © 2025 General Assembly

The scale of the *x*-axis is way larger! We need to rescale!



 © 2025 General Assembly

K-Means

Let's see it for ourselves! Parts 1–4

→ Open **02a-kmeans-NOSLIDES-solutions.ipynb**



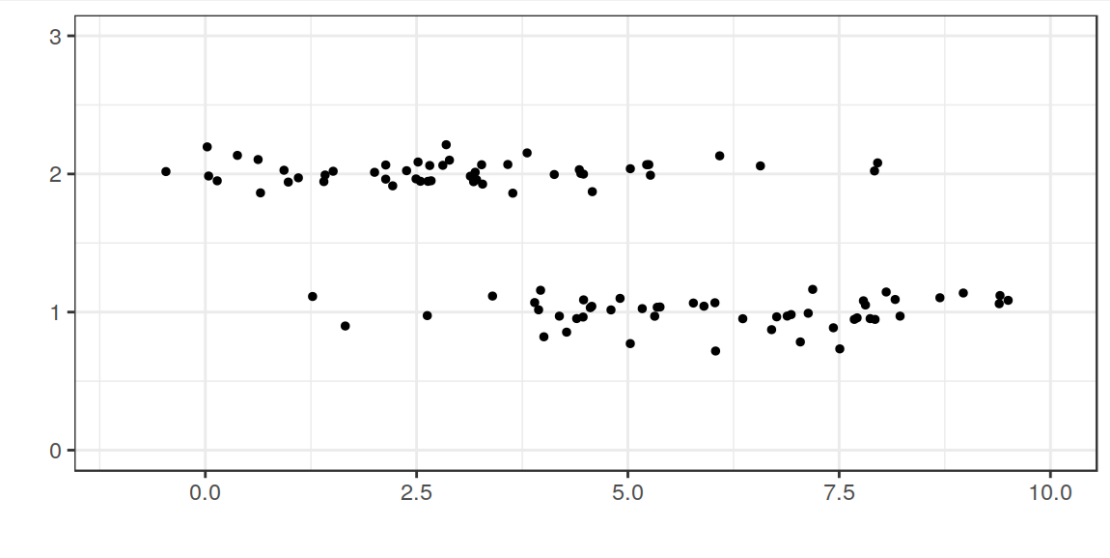
 © 2025 General Assembly

Break time



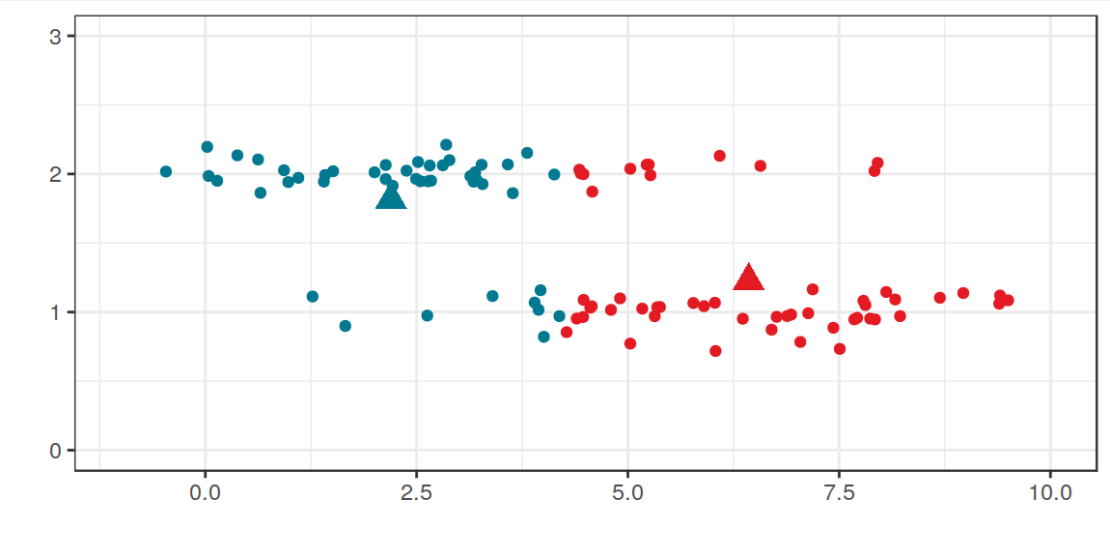
How good is our clustering?

Clustering Doesn’t Always Work



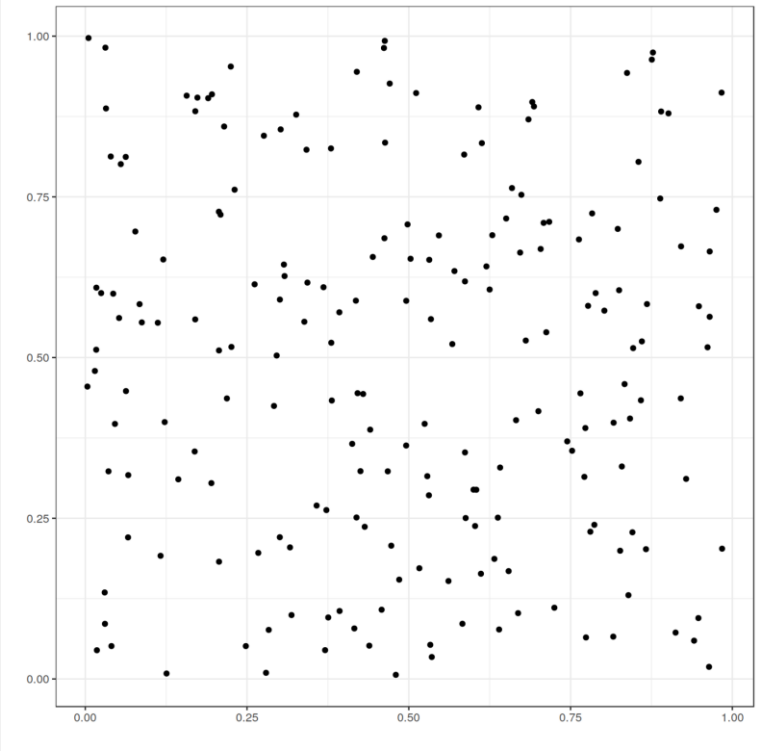
 © 2025 General Assembly

Clustering Doesn't Always Work



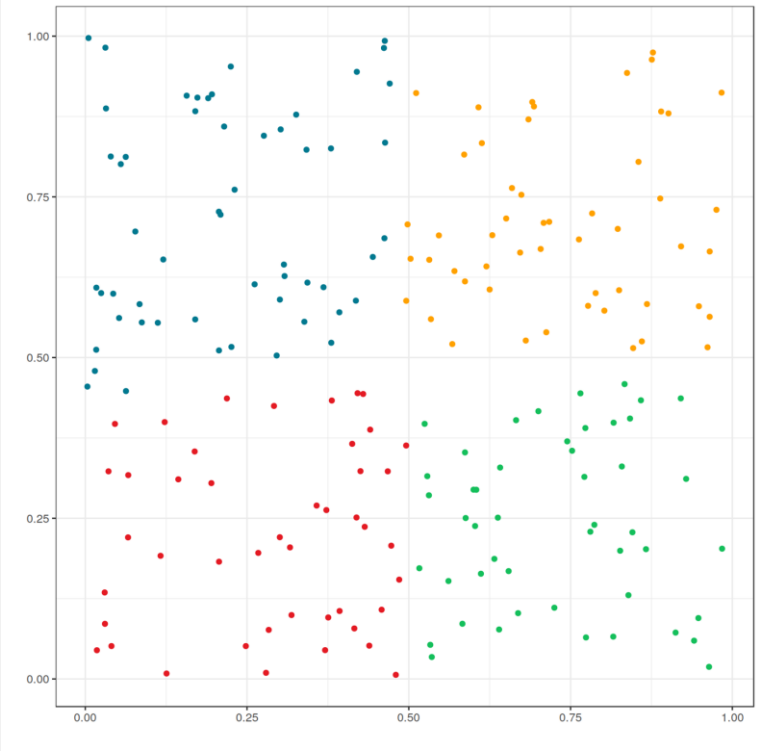
 © 2025 General Assembly

Clustering Doesn’t Always Work



 © 2025 General Assembly

Clustering Doesn’t Always Work



 © 2025 General Assembly

A Bad Time To Cluster

As we just saw, it is possible for a data set to have no discernible “grouping” structure.

If this is the case, **clustering won’t fail**! It will segment your data into *some groups*, but they won’t be very separate or useful!

What’s worse, we usually **can’t even tell if this is the case!** Our data is very rarely only two-dimensional. We might have thousands of columns, as we will soon see…



 © 2025 General Assembly

How To Know You've Done a "Good Job" Clustering

If we can't see our clusters, we can always use a few **clustering metrics** to give us a single number that summarizes our analysis. We'll focus on two:

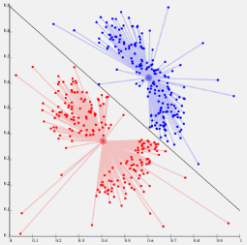
1. **Inertia**: The sum squared distances of each point to its centroid.
2. **Silhouette score**: Measures how separate and cohesive our clusters are.



Source

 © 2025 General Assembly

Inertia (aka "Within-Cluster Sum of Squares")



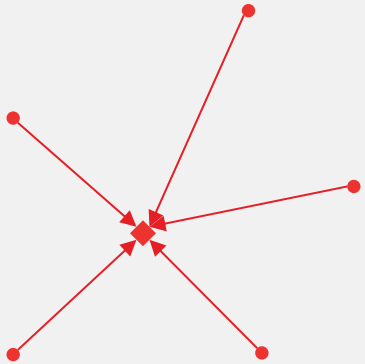
Inertia represents the sum squared distances each point is from its centroid. **Low inertia = dense cluster**.

$$\sum \|x_{ij} - \mu_j\|^2$$

Observation *i* from cluster *j*

Centroid of cluster j

Distance between them



 © 2025 General Assembly

Source

Example: calculating inertia

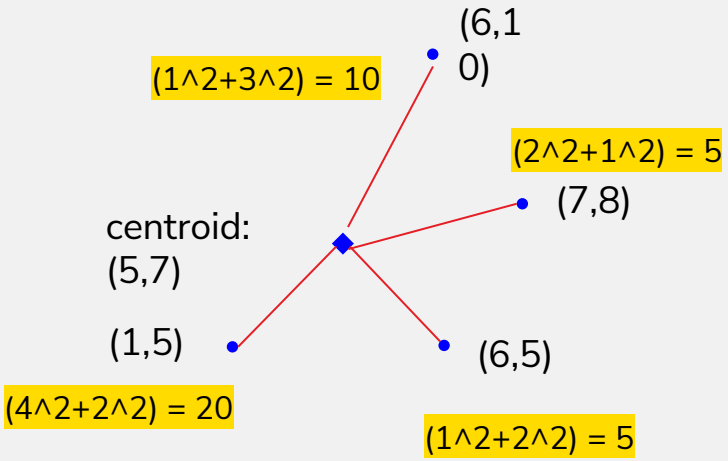
This represents one cluster. To get overall inertia, we sum the inertia for all clusters.

$$\sum \|x_{ij} - \mu_j\|^2$$

Observation *i* from cluster *j*

Centroid of cluster j

Distance between them



Answer: 10 + 5 + 20 + 5 = 40

 © 2025 General Assembly

The Problems With Inertia

There are two problems with using inertia:

- It is uninterpretable to humans.
- As long as we keep increasing the number of clusters, inertia will always decrease. That is why we use the Elbow method.

 © 2025 General Assembly

Another measure: the Silhouette Score

The silhouette score measures the overall **cohesion** and **separation** of our clusters. That is, we believe that:

- Clusters should be tightly packed together, and
- Clusters should be far apart from other clusters.

It is computed as follows...

 © 2025 General Assembly

Silhouette Score

For each observation i, we compute:

$$s_i = \frac{b_i - a_i}{\max\{a_i, b_i\}}$$

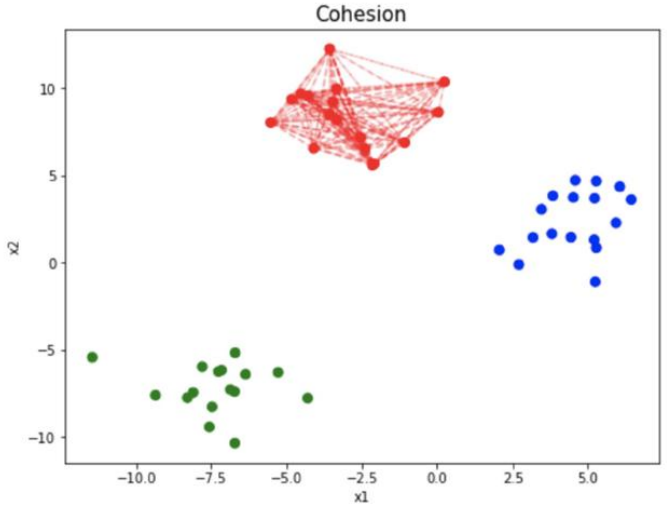
a_i = **Cohesion**: The average distance from point i to points in the **same cluster.**

b_i = **Separation:** The average distance from point i to points in the **next nearest cluster.**

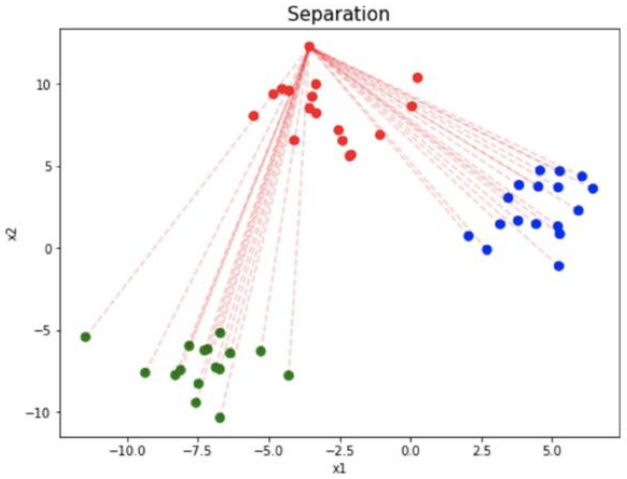
 © 2025 General Assembly

How is the Silhouette Score calculated?

Cohesion - Average Distance of Points within Clusters



Separation - Average Distance of Points in One Cluster to Points in Other Clusters



 © 2025 General Assembly

Silhouette Score

We then average each s_i. Notice that if our clustering is **good**,

large

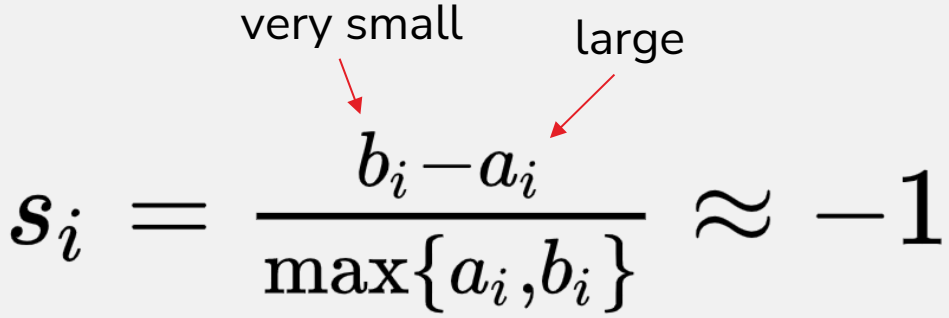
very small

$$s_i = \frac{b_i - a_i}{\max\{a_i, b_i\}} \approx 1$$

 © 2025 General Assembly

Silhouette Score

Conversely, if our clustering is **bad**,



 © 2025 General Assembly

Silhouette Score

So the silhouette score ranges between -1 and 1, where:

- **1** = Dense, well-separated clusters
- **0** = Overlapping clusters (points on the boundary)
- **-1** = Points closer to another cluster (likely misassigned)

Note: to get a score for all clusters rather than for a particular point, we average over all points to judge the cluster algorithm.

 © 2025 General Assembly

K-Means evaluation

Let's see it for ourselves! Parts 5–8

→ Open **02a-kmeans-NOSLIDES-solutions.ipynb**



 © 2025 General Assembly

Key considerations with K-Means

Key assumptions of K-Means

- **k** is the **correct number of clusters**
- The data is **isotropically distributed (circular/spherical distribution)**
- Clusters have **a similar spread (variance)**
- Clusters are **roughly the same size**

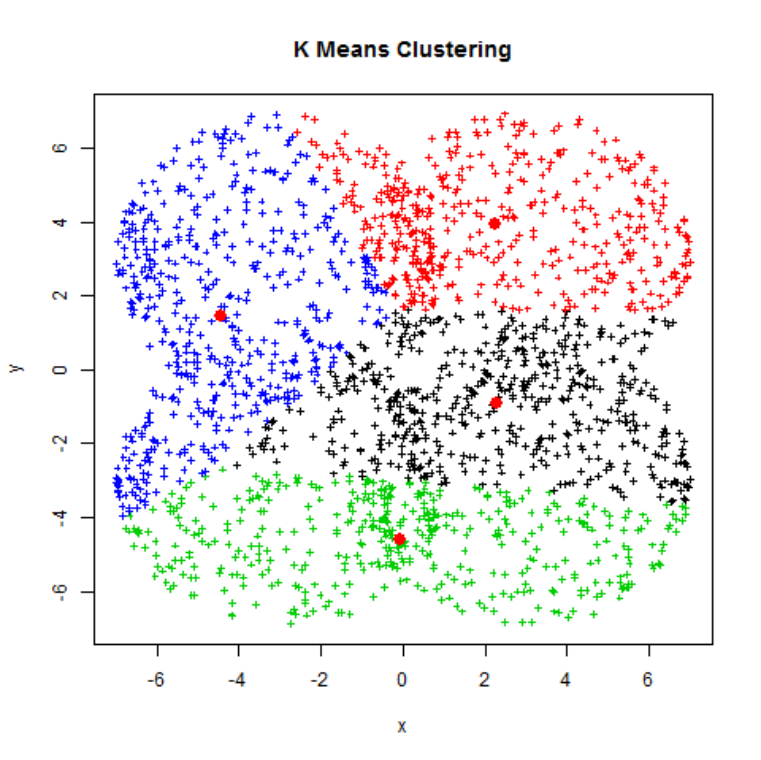
Look at these resources to see counterexamples / cases where assumptions are not met:

- *Variance Explained*
- *scikit-Learn*

 © 2025 General Assembly

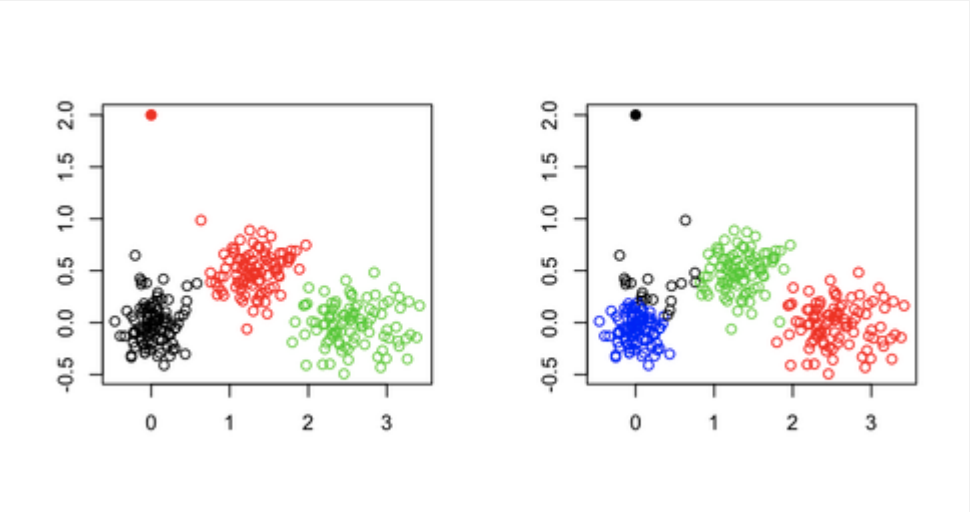
A note on K-Means Convergence

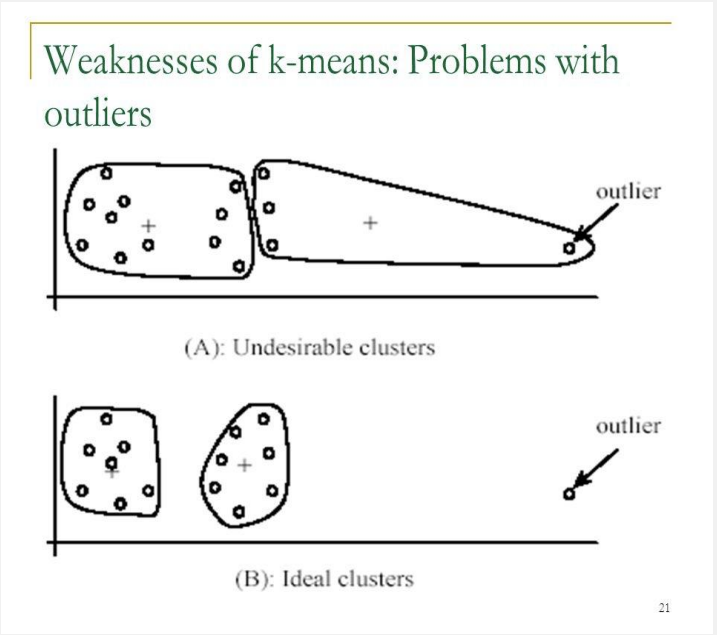
- In general, k-means will converge to a solution and return a partition of k clusters, even if no natural clusters exist in the data.
- It's entirely possible – in fact, common – that the clusters do not mean anything at all.



 © 2025 General Assembly

K-Means Sensitivities: Outliers

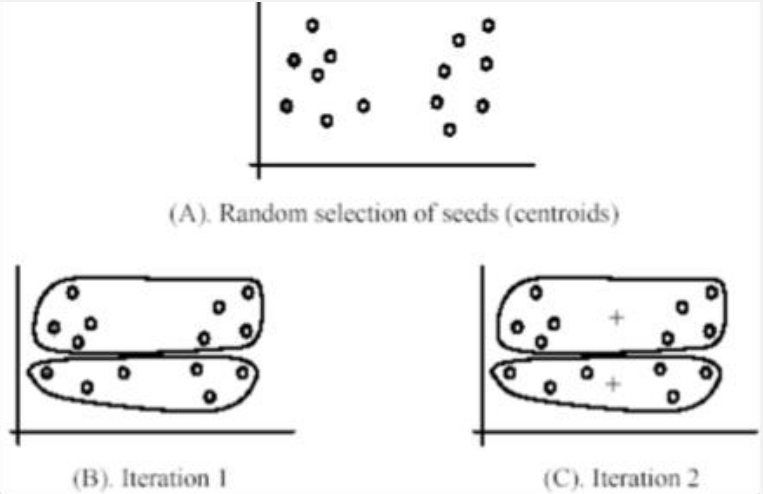




 © 2025 General Assembly

K-Means Sensitivities: Centroid Initialization

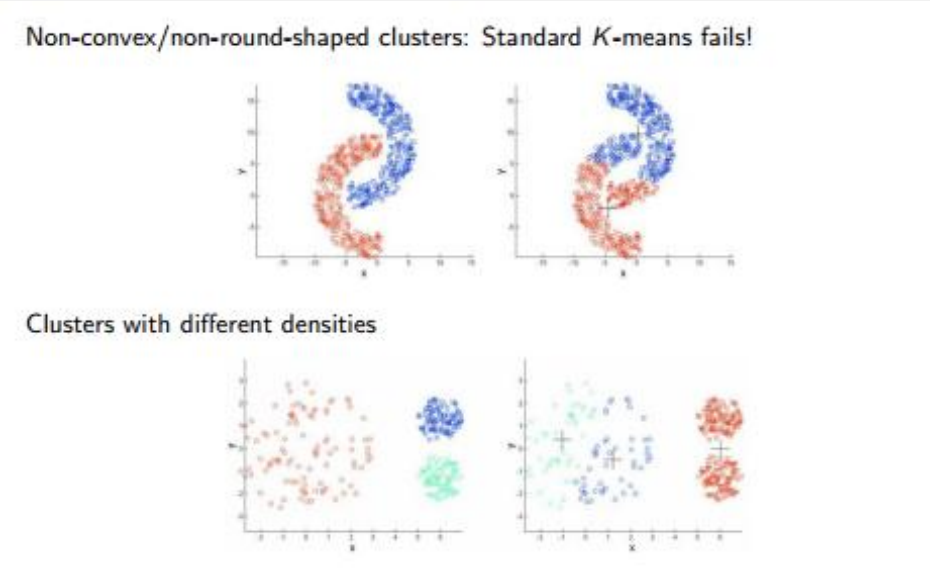
K-Means is sensitive to centroid initialization



 © 2025 General Assembly

K-Means Sensitivities: Non-Linearities

K-Means is sensitive to non-linearities



 © 2025 General Assembly

Bring It Home

K-Means Strengths & Weaknesses

STRENGTHS	WEAKNESSES
→ k-means is popular because it's simple and computationally efficient. → Easy to see results / intuitive. → It always converges (but possibly to a local optimum, so run it several times).	→ K-Means is highly scale dependent and isn't suitable for data of varying shapes and densities. → Evaluating results is more subjective, requiring much more human evaluation than trusted metrics. → It is sensitive to outliers, centroid initialization and non-linearities.

 © 2024 General Assembly

When is *K*-Means good?

k-means is best when:

- Your clusters are blob-shaped (as opposed to U-shaped)
- Your clusters are more circular (as opposed to a stripe shape)
- Your blob-shaped, circular clusters are approximately the same size

These conditions are pretty much never met with real data. Nonetheless, *k*-means usually performs really well on imperfect data!

 © 2025 General Assembly

Key takeaways



Clustering is useful for finding groups in data

It can be useful for lots of tasks, such as market segmentation and recommendations



K-Means is simple and easy to implement

You just need to select the number of clusters and can evaluate with inertia and silhouette scores



You need to be careful of assumptions

Optimal k is hard to evaluate and clusters are assumed to be the same size and isotropically distributed

 © 2024 General Assembly


GA
GENERAL ASSEMBLY